# PAPER CITATION CLASSIFICATION

## Abstract

This project investigates whether information associated with a research paper can be used to classify its citation impact. Each paper is assigned to one of three categories—Low, Medium, or High—based on its citation count. The model inputs include the paper title and abstract, publication year, author count, and primary research field. The implementation combines natural language processing with structured metadata preprocessing. TF-IDF is used to represent textual content, numeric metadata is standardized, and categorical field information is one-hot encoded. Three machine-learning approaches are evaluated: Multinomial Naive Bayes, Random Forest, and a multilayer perceptron (MLP) neural network. On the 18,000-record test set, the neural network achieves the highest accuracy (40.70%), while Random Forest achieves the highest weighted F1-score (0.3936). The results indicate that citation classification is challenging using the selected features alone and that future work should investigate richer features and more controlled model comparisons.

## 1. Introduction

Research papers receive different numbers of citations, and citation counts can be treated as one imperfect indicator of how frequently later work refers to a publication. Estimating a paper's likely citation group may help researchers study patterns associated with scholarly attention. However, citation counts are influenced by several factors, including publication age, research discipline, venue, accessibility, and the size of the research community. They are also highly skewed: many papers receive relatively few citations, while a smaller number receive very many.

The project frames the task as supervised multi-class classification rather than exact numerical prediction. A model receives features describing a paper and predicts whether it belongs to the Low, Medium, or High citation class. The project follows the broad approach of the Stanford CS229 report "Paper Citation Classification" (Bui, Saavedra, and Liu, 2019), but uses a different 120,000-record OpenAlex-derived dataset and expands the input information to include abstracts and author-count/field metadata.

## 2. Problem Statement and Objectives

Given metadata and text describing an academic paper, predict its citation class (Low, Medium, or High). The citation count is used to construct the class label but must not be included as an input feature, because doing so would reveal the information from which the target is derived and create target leakage.

The main objectives are:

- Prepare a research-paper dataset with textual and structured features.
- Convert title and abstract text into numerical TF-IDF features.
- Preprocess numeric and categorical metadata using suitable transformations.
- Train and compare Naive Bayes, Random Forest, and MLP Neural Network models.
- Evaluate models using accuracy, precision, recall, F1-score, and confusion matrices.
- Identify limitations and potential improvements for citation-impact classification.

# 3. Dataset Description

The working dataset contains 120,000 academic-paper records collected from OpenAlex-derived data and stored in paper_citation_classification_120k.csv. The project's selected fields are summarized below. Confirm the final source URL and exact dataset version in the repository README before submission.

| Column | Role | Description |
|---|---|---|
| paper_id | Identifier | Identifier for the paper/work. |
| title | Text input | Title of the academic paper. |
| abstract | Text input | Abstract describing the paper's research. |
| publication_year | Numeric input | Year in which the paper was published. |
| author_count | Numeric input | Number of authors associated with the paper. |
| primary_field | Categorical input | Main research field or topic category. |
| citation_count | Target source only | Number of citations; used to derive citation_class, not fed to the models. |

## 3.1 Target Class Construction

The dataset was divided into 85% training data and 15% testing data: 102,000 records for training and 18,000 records for testing. The project uses citation-count quantiles derived from the training data to define three groups. The thresholds recorded in the working notebook are Low ≤ 918, Medium = 919–1,384, and High ≥ 1,385. These boundaries should be verified against the exact code used to create citation_class before final submission, because the final threshold values must match the notebook and saved dataset.

The training label distribution reported by the notebook was approximately balanced: Medium 34,637 records, Low 33,710 records, and High 33,653 records. This is useful because a model is less likely to achieve a seemingly good score simply by predicting one dominant class. The test-set class supports were High 6,052, Low 5,738, and Medium 6,210.

# 4. Data Preparation and Feature Engineering

## 4.1 Text Combination

For each paper, the title and abstract are combined into a single text field called paper_text. This allows the model to use both the short title and the longer abstract as evidence. Missing text should be converted to an empty string before concatenation so that string operations do not fail.

## 4.2 TF-IDF Text Representation

Machine-learning classifiers require numerical inputs, so the text is transformed using Term Frequency–Inverse Document Frequency (TF-IDF). TF-IDF gives greater weight to terms that are informative for particular documents and reduces the influence of terms that occur very frequently throughout the corpus. The configured vectorizer uses a maximum of 20,000 features, English stop-word removal, unigrams and bigrams, a minimum document frequency of three, and sublinear term-frequency scaling. Unigrams are individual terms, while bigrams are two-word sequences that can preserve simple phrases.

The vectorizer is fitted only on training text and then applied to test text with the same learned vocabulary. This is important to prevent information from the test set influencing feature construction. The notebook reports TF-IDF matrices with shapes (102000, 20000) for training and (18000, 20000) for testing.

## 4.3 Metadata Processing

The structured features are processed separately from text. StandardScaler standardizes publication_year and author_count using statistics learned from training data. OneHotEncoder converts primary_field categories into binary columns and ignores unseen categories in the test set. ColumnTransformer combines these steps into a single preprocessing pipeline.

The combined training matrix has shape (102000, 20028), and the test matrix has shape (18000, 20028). These 20,028 columns consist of 20,000 TF-IDF text features and 28 processed metadata features, according to the notebook output.

### 4.4 Dimensionality Reduction for the Neural Network

The combined matrix is high-dimensional and sparse. TruncatedSVD is used to reduce it to 200 components for the neural-network experiment. This yields dense training and test arrays of shapes (102000, 200) and (18000, 200). The arrays are explicitly converted to float32. The target labels are encoded as numeric values for MLPClassifier and mapped back to their class names for reporting.

## 5. Machine-Learning Models

### 5.1 Multinomial Naive Bayes

Multinomial Naive Bayes is used as a baseline text classifier. It estimates class probabilities from term counts or non-negative text features under a conditional-independence assumption. In this project it is trained on the TF-IDF text matrix and predicts Low, Medium, or High. It is relatively fast and provides a useful baseline, but it does not use the structured metadata in the shown training cell.

### 5.2 Random Forest

Random Forest is an ensemble of decision trees. Each tree learns decision rules from a sampled view of the data, and their outputs are combined for a final prediction. The notebook uses 100 trees, a maximum depth of 20, a minimum of five samples per leaf, and a fixed random seed. It is trained on the combined text-and-metadata representation. Random forests can model nonlinear patterns, but very high-dimensional text features can make training expensive and may lead to limited generalization.

### 5.3 MLP Neural Network

The MLPClassifier is a feed-forward neural network with one hidden layer of 64 units and ReLU activation. It uses the Adam optimizer, a batch size of 256, a learning rate of 0.001, early stopping, and up to 15 iterations. It is trained on the 200-component SVD representation. The notebook initially encountered a type-related error when the reduced features were passed to the model; converting the arrays to float32 and encoding the target labels resolved the problem, and the model subsequently trained and produced predictions.

## 6. Experimental Setup and Evaluation

The dataset split is 85:15, with 102,000 training examples and 18,000 held-out test examples. The test set is used to compare predictions with known citation classes. Accuracy measures the proportion of all predictions that are correct. Precision measures how often predictions for a class are correct; recall measures how many actual members of a class are recovered. The F1-score is the harmonic mean of precision and recall. Weighted averages account for each class's support when combining class-level scores.

Confusion matrices display actual classes in rows and predicted classes in columns. Diagonal cells are correct predictions, while off-diagonal cells represent misclassifications. They help reveal whether a model confuses Low with Medium, Medium with High, or other class pairs.

## 7. Results

The notebook reports the following test-set metrics:

| Model | Accuracy | Weighted Precision | Weighted Recall | Weighted F1 |
|---|---|---|---|---|
| Naive Bayes | 0.3936 | 0.3853 | 0.3936 | 0.3781 |
| Random Forest | 0.3968 | 0.3980 | 0.3968 | 0.3936 |
| Neural Network | 0.4070 | 0.3989 | 0.4070 | 0.3893 |

### 7.1 Interpretation

The Neural Network has the highest accuracy at 0.4070 (40.70%) and the highest weighted recall at 0.4070. Random Forest has the highest weighted F1-score at 0.3936 and weighted precision at 0.3980 is slightly below the neural

network's 0.3989. Naive Bayes is the lowest of the three by accuracy and weighted F1-score. The accuracy difference between the best and worst models is about 1.34 percentage points, so the improvement is modest rather than dramatic.

The confusion matrices show substantial mixing between classes. For Naive Bayes, many Medium examples are predicted as High, and Medium is the least reliably identified class in the per-class report (F1 = 0.26). Random Forest improves Medium recall to 0.44, although it still misclassifies many Low and Medium examples. The neural network correctly classifies more Low examples than the other models, but its Medium-class recall remains limited. The results suggest that text and the selected metadata provide some signal, but are not sufficient for highly accurate citation-class prediction.

## 8. Discussion

The results should be interpreted in light of the feature and model differences. Naive Bayes uses text features only, while Random Forest uses the combined text-and-metadata representation, and the neural network uses an SVD-reduced version of the combined representation. Therefore, the experiment compares complete pipelines rather than isolating the effect of the algorithm alone. A more controlled comparison would train all models on the same feature set where practical, while also considering the constraints of sparse high-dimensional text data.

Citation count also depends on paper age: older papers generally have had more time to accumulate citations. Publication year is included as a feature, but a simple year value may not capture all time-related effects. The target bins are derived from the training distribution, which helps create usable class sizes, but quantile thresholds can vary with the sample and data source. Results should not be interpreted as causal evidence that a title, abstract, or author count causes citation impact.

## 9. Limitations and Future Work

- Moderate predictive performance: the best test accuracy is approximately 40.7%, so predictions should be treated as estimates.
- Limited metadata: venue, field-specific citation practices, author experience, open-access status, references, and other variables may be useful if available and ethically appropriate.
- Different feature pipelines: model results are not a pure algorithm-only comparison because input representations differ.
- Text representation: TF-IDF captures term occurrence patterns but does not fully understand semantic meaning or context.
- Class thresholds: the final thresholds and label distribution must be consistent with the exact dataset and code used for submission.
- Potential next steps include tuning hyperparameters, examining per-class metrics, testing a consistent feature pipeline, comparing alternative text representations, and documenting reproducibility.

## 10. Conclusion

This project developed a supervised machine-learning pipeline for classifying academic papers into Low, Medium, and High citation groups. It combined title-and-abstract TF-IDF features with publication-year, author-count, and research-field metadata. Naive Bayes, Random Forest, and an MLP Neural Network were trained and evaluated using an 85:15 split and multiple metrics. The Neural Network achieved the highest accuracy (40.70%), while Random Forest achieved the highest weighted F1-score (0.3936). The modest scores and confusion-matrix errors demonstrate that citation classification is challenging and that further improvements require better features, careful experimental control, and additional validation.

## References

Bui, C., Saavedra, C., & Liu, S. (2019). Paper Citation Classification. Stanford CS229 Machine Learning Final Report.

OpenAlex. Academic works metadata and citation information. The project uses an OpenAlex-derived 120,000-record subset; include the exact dataset URL/version in the final repository README.

## Appendix: Reproducibility Checklist

- Include the dataset source, license, and record count in README.
- Ensure the CSV and notebook use the same citation-class thresholds.
- Document package requirements and instructions to run the notebook.
- Include the results table and confusion-matrix outputs in the project repository.
- Do not publish API keys or other credentials in notebooks or GitHub.